What is Encoding

মেশিন লার্নিং মডেল সাধারণত সংখ্যাভিত্তিক (Numerical) ডেটা বুঝতে পারে। কিন্তু বাস্তব জীবনের অনেক ডেটাসেটে আমরা টেক্সট বা ক্যাটেগরিক্যাল ডেটা পাই, যেমন:

Gender → Male, Female
City → Dhaka, Chittagong, Khulna
Education → SSC, HSC, Bachelor

মেশিন এই ধরনের টেক্সট সরাসরি বুঝতে পারে না। তাই ক্যাটেগরিক্যাল ডেটাকে সংখ্যায় রূপান্তর করার প্রক্রিয়াকে Encoding বলা হয়।

Encoding করার মাধ্যমে মডেল ডেটার ভেতরের সম্পর্ক বিশ্লেষণ করতে পারে।

উদাহরণ:

Gender
Male
Female
Male

Encoding করার পর:

Gender
1
0
1

এখানে:

Male → 1
Female → 0

Encoding মূলত দুই ধরনের ডেটার ক্ষেত্রে বেশি ব্যবহার করা হয়:

Ordinal Data
Nominal Data

Ordinal Data:

যেখানে একটি নির্দিষ্ট ক্রম বা Order থাকে।

উদাহরণ:

Low
Medium
High

Nominal Data:

যেখানে কোনো Order থাকে না।

উদাহরণ:

Red
Blue

Green

Encoding এর প্রয়োজনীয়তা:

Machine Learning model numeric input চায়
Data processing সহজ হয়
Model দ্রুত train করতে পারে
অনেক algorithm categorical value handle করতে পারে না

Encoding এর কিছু জনপ্রিয় পদ্ধতি:

Ordinal Encoding
One Hot Encoding
Label Encoding
Frequency Encoding
Target Encoding

Ordinal Encoding

Ordinal Encoding হলো এমন একটি Encoding Technique যেখানে ক্যাটেগরিক্যাল ডেটাকে নির্দিষ্ট ক্রম (Order) অনুযায়ী সংখ্যায় রূপান্তর করা হয়।
এই Encoding শুধুমাত্র সেইসব Feature এর ক্ষেত্রে ব্যবহার করা হয় যেখানে category গুলোর মধ্যে একটি meaningful order বা ranking থাকে।
উদাহরণ:

Low
Medium
High

এখানে একটি স্বাভাবিক ক্রম রয়েছে:
Low < Medium < High
তাই আমরা এগুলোকে এভাবে encode করতে পারি:

Low → 0
Medium → 1
High → 2

Machine Learning model তখন বুঝতে পারে যে High এর মান Medium থেকে বড় এবং Medium এর মান Low থেকে বড়।
কেন **Ordinal Encoding** ব্যবহার করা হয়
অনেক Dataset এ এমন Feature থাকে যেখানে category গুলোর মধ্যে level বা ranking থাকে।
উদাহরণ:

Education Level
Customer Satisfaction
Shirt Size
Experience Level

এসব ক্ষেত্রে Ordinal Encoding খুব কার্যকর।
Ordinal Data এর উদাহরণ

Feature Categories	
Education	SSC, HSC, Bachelor, Masters
Satisfaction	Poor, Average, Good
Size	Small, Medium, Large
Experience	Beginner, Intermediate, Expert

এখানে প্রতিটি category এর একটি logical order রয়েছে।

Ordinal Encoding কীভাবে কাজ করে

ধরা যাক একটি Feature আছে:

Performance
Poor
Average
Good
Excellent

Encoding করার পর:

Performance
0
1
2
3

এখানে সংখ্যাগুলো category এর order প্রকাশ করছে।

Python এ **Ordinal Encoding**

প্রথমে প্রয়োজনীয় Library import করতে হবে।

```
import pandas as pd
```

এখন একটি DataFrame তৈরি করি।

```
df = pd.DataFrame({
    'Performance': ['Poor', 'Average', 'Good', 'Excellent', 'Good']
})

print(df)
```

Output:

```
  Performance
0        Poor
1     Average
2        Good
```

```
3    Excellent
4         Good
```

এখন আমরা একটি mapping dictionary তৈরি করব।

```
encoding_map = {
    'Poor': 0,
    'Average': 1,
    'Good': 2,
    'Excellent': 3
}
```

এরপর map() function ব্যবহার করে encoding করব।

```
df['Performance'] = df['Performance'].map(encoding_map)

print(df)
```

Output:

```
   Performance
0            0
1            1
2            2
3            3
4            2
```

Education Level Encoding Example

```
df = pd.DataFrame({
    'Education': ['SSC', 'HSC', 'Bachelor', 'Masters', 'HSC']
})

education_order = {
    'SSC': 0,
    'HSC': 1,
    'Bachelor': 2,
    'Masters': 3
}

df['Education'] = df['Education'].map(education_order)
```

```
13
14print(df)
```

Output:

```
   Education
0          0
1          1
2          2
3          3
4          1
```

Scikit-Learn ব্যবহার করে **Ordinal Encoding**
Scikit-Learn এ OrdinalEncoder ব্যবহার করেও encoding করা যায়।

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder
```

Dataset তৈরি করি:

```
df = pd.DataFrame({
    'Size': ['Small', 'Medium', 'Large', 'Medium', 'Small']
})

print(df)
```

Encoder তৈরি:

```
encoder = OrdinalEncoder(categories=[['Small', 'Medium', 'Large']])
```

Encoding:

```
df['Size'] = encoder.fit_transform(df[['Size']])

print(df)
```

Output:

```
   Size
0   0.0
1   1.0
2   2.0
3   1.0
4   0.0
```

Ordinal Encoding এর সুবিধা

সহজ এবং দ্রুত Encoding Technique
কম Memory ব্যবহার করে
Ordered data খুব সুন্দরভাবে represent করতে পারে
Machine Learning model এর জন্য numeric format তৈরি করে

Ordinal Encoding এর অসুবিধা
যদি কোনো Feature এর মধ্যে প্রকৃতপক্ষে order না থাকে, তাহলে Ordinal Encoding ব্যবহার করা উচিত নয়।
উদাহরণ:

Red
Blue
Green

এখানে কোনো ranking নেই।
যদি encode করি:

Red → 0
Blue → 1
Green → 2

তাহলে model ভুলভাবে ধরে নিতে পারে Green > Blue > Red।
এটি model এর performance খারাপ করতে পারে।
তাই শুধুমাত্র Ordered categorical data এর ক্ষেত্রেই Ordinal Encoding ব্যবহার করা উচিত।

Nominal Value Encoding

Nominal Value Encoding হলো এমন একটি Encoding প্রক্রিয়া যেখানে এমন categorical data কে numerical format এ রূপান্তর করা হয় যাদের মধ্যে কোনো ধরনের order বা ranking থাকে না।
অর্থাৎ category গুলোর মধ্যে বড়-ছোট বা আগে-পরে কোনো সম্পর্ক থাকে না।
উদাহরণ:

Red
Blue
Green

এখানে কোনো category অন্য category থেকে বড় বা ছোট নয়। তাই এগুলোকে Ordinal Encoding এর মতো encode করা ঠিক নয়।
এই ধরনের data কে Nominal Data বলা হয়।
Nominal Data কী
যেসব categorical data এর মধ্যে কোনো logical order থাকে না, সেগুলোকে Nominal Data বলা হয়।
উদাহরণ:

Feature **Categories**	
Color	Red, Blue, Green
City	Dhaka, Chittagong, Khulna

Gender	Male, Female
Blood Group	A, B, AB, O

এখানে category গুলোর মধ্যে কোনো ranking নেই।

Nominal Value Encoding কেন প্রয়োজন

Machine Learning model সাধারণত text data বুঝতে পারে না।

যেমন:

```
1Red
2Blue
3Green
```

এগুলোকে model এর জন্য numerical format এ রূপান্তর করতে হয়।

তবে এখানে একটি গুরুত্বপূর্ণ বিষয় হলো:

Encoding করার সময় model যেন ভুলভাবে কোনো order ধরে না নেয়।

কারণ:

```
1Red = 0
2Blue = 1
3Green = 2
```

এভাবে encode করলে model ভাবতে পারে:

Green > Blue > Red

কিন্তু বাস্তবে এমন কোনো সম্পর্ক নেই।

তাই Nominal Data এর জন্য বিশেষ Encoding technique ব্যবহার করা হয়।

Nominal Encoding এর জনপ্রিয় পদ্ধতি

Nominal Data encode করার জন্য কয়েকটি জনপ্রিয় পদ্ধতি হলো:

- One Hot Encoding
- Label Encoding
- Frequency Encoding
- Binary Encoding

এই মডিউলে আমরা মূলত One Hot Encoding এবং Label Encoding নিয়ে আলোচনা করব।

ভুল **Encoding** এর সমস্যা

ধরা যাক একটি Dataset এ City column আছে:

City
Dhaka
Khulna
Chittagong

যদি encode করি:

```
1Dhaka = 0
2Khulna = 1
3Chittagong = 2
```

তাহলে কিছু Machine Learning algorithm ধরে নিতে পারে:

```
Chittagong > Khulna > Dhaka
```

এটি ভুল interpretation তৈরি করতে পারে।
বিশেষ করে:

- Linear Regression
- Logistic Regression
- KNN
- Neural Network

এই ধরনের model এ সমস্যা হতে পারে।
Nominal Data Handling Example
প্রথমে একটি Dataset তৈরি করি।

```
import pandas as pd

df = pd.DataFrame({
    'Color': ['Red', 'Blue', 'Green', 'Blue', 'Red']
})

print(df)
```

Output:

```
   Color
0    Red
1   Blue
2  Green
3   Blue
4    Red
```

এখন আমরা unique category গুলো দেখতে পারি।

```
print(df['Color'].unique())
```

Output:

```
['Red' 'Blue' 'Green']
```

এখানে কোনো order নেই। তাই Ordinal Encoding ব্যবহার করা ঠিক হবে না।
Nominal Data এবং **Ordinal Data** এর পার্থক্য

Nominal Data **Ordinal Data**	
কোনো order নেই	নির্দিষ্ট order আছে

Example: Color	Example: Education Level
Ranking সম্ভব নয়	Ranking সম্ভব
One Hot Encoding বেশি ব্যবহৃত হয়	Ordinal Encoding বেশি ব্যবহৃত হয়

বাস্তব জীবনে **Nominal Data** এর উদাহরণ
অনেক Dataset এ Nominal Feature দেখা যায়।
উদাহরণ:

- Country
- Browser Name
- Department Name
- Product Category
- Blood Group
- Car Brand

এসব ক্ষেত্রেই category গুলোর মধ্যে কোনো mathematical relationship থাকে না।
Nominal Encoding এর মূল লক্ষ্য
Nominal Encoding এর প্রধান উদ্দেশ্য হলো:

- Categorical data কে numerical format এ রূপান্তর করা
- কোনো false ordering তৈরি না করা
- Machine Learning model এর জন্য উপযুক্ত input তৈরি করা

পরবর্তী মডিউলে আমরা One Hot Encoding নিয়ে বিস্তারিত আলোচনা করব, যা Nominal Data handle করার সবচেয়ে জনপ্রিয় পদ্ধতিগুলোর একটি।

One Hot Encoding

One Hot Encoding হলো একটি জনপ্রিয় encoding technique যেখানে categorical data কে এমনভাবে numerical format এ রূপান্তর করা হয় যাতে প্রতিটি category আলাদা আলাদা binary column হিসেবে represent করা হয়।
এই পদ্ধতিতে প্রতিটি category এর জন্য একটি নতুন column তৈরি হয় এবং সেই category থাকলে মান হয় 1, না থাকলে মান হয় 0।
One Hot Encoding বিশেষ করে Nominal Data এর ক্ষেত্রে ব্যবহার করা হয়, যেখানে কোনো order বা ranking থাকে না।
One Hot Encoding কীভাবে কাজ করে
ধরা যাক একটি feature আছে:

City
Dhaka
Khulna
Chittagong

One Hot Encoding করার পর এটি এমন হবে:

Dhaka	Khulna	Chittagong
1	0	0
0	1	0
0	0	1

এখানে প্রতিটি category আলাদা column এ convert হয়েছে।

One Hot Encoding কেন ব্যবহার করা হয়

- কোনো false ordering তৈরি হয় না
- Nominal data handle করার জন্য সবচেয়ে safe method
- Machine Learning model এর জন্য clean representation তৈরি করে
- Linear models এবং neural networks এর জন্য খুব effective

Ordinal Encoding বনাম **One Hot Encoding**

Ordinal Encoding এ numerical order তৈরি হয়, কিন্তু One Hot Encoding এ কোনো order থাকে না।

উদাহরণ:

Ordinal Encoding:

- Red → 0
- Blue → 1
- Green → 2

One Hot Encoding:

- Red → (1,0,0)
- Blue → (0,1,0)
- Green → (0,0,1)

Scikit-Learn দিয়ে **One Hot Encoding**

Scikit-Learn এর OneHotEncoder ব্যবহার করে সহজেই encoding করা যায়।

প্রথমে প্রয়োজনীয় library import করি:

```
1from sklearn.preprocessing import OneHotEncoder
```

```
2import pandas as pd
```

ধরা যাক আমাদের dataset এ Embarked feature আছে।

```
1embarked_ohe =
OneHotEncoder(sparse_output=False).set_output(transform='pandas')
```

এখানে:

- sparse_output=False → dense dataframe তৈরি করে
- set_output(transform='pandas') → output কে pandas DataFrame হিসেবে রাখে

Model Fit করা

```
1embarked_ohe.fit(X_train[['Embarked']])
```

এটি training data থেকে unique categories শিখে নেয়।

Training Data Transform করা

```
1encoded_df = embarked_ohe.transform(X_train[['Embarked']])


2X_train = pd.concat([X_train, encoded_df], axis=1)
```

এখানে:

- নতুন encoded columns তৈরি হয়
- পুরাতন X_train এর সাথে merge করা হয়

Test Data Transform করা

```
1encoded_df = embarked_ohe.transform(X_test[['Embarked']])


2X_test = pd.concat([X_test, encoded_df], axis=1)
```

গুরুত্বপূর্ণ বিষয় হলো:

- fit() শুধুমাত্র training data তে করা হয়
- transform() train এবং test দুই জায়গায় ব্যবহার করা হয়

One Hot Encoding এর পরে **unnecessary columns remove** করা

```
1X_train.drop(['Pclass', 'Sex', 'Embarked'], axis=1, inplace=True)


2X_test.drop(['Pclass', 'Sex', 'Embarked'], axis=1, inplace=True)
```

এখানে মূল categorical columns গুলো drop করা হয়েছে, কারণ এগুলোর encoded version ইতিমধ্যে তৈরি হয়েছে।

Real Dataset Example (Embarked Encoding)

ধরা যাক:

Embarked
S
C
Q

Encoding এর পর:

Embarked_C	**Embarked_Q**	**Embarked_S**
0	0	1
1	0	0
0	1	0

One Hot Encoding এর সুবিধা

No ordering issue
Better performance for many ML models
Easy interpretation for binary features
Works well for small to medium number of categories

One Hot Encoding এর অসুবিধা

High dimensionality (অনেক category হলে অনেক columns তৈরি হয়)
Sparse matrix issue (memory usage বাড়ে)
Large datasets এ computational cost বাড়তে পারে

উদাহরণ:
যদি কোনো feature এ 100 categories থাকে, তাহলে 100 নতুন column তৈরি হবে।
Real Dataset Context (Cabin Feature Insight)

```
1X_train['Cabin'].value_counts()
```

Cabin feature সাধারণত অনেক unique value থাকে, তাই One Hot Encoding করলে খুব বেশি columns তৈরি হতে পারে।
এই ধরনের ক্ষেত্রে সাধারণত:

Feature engineering করা হয়
Rare category grouping করা হয়
অথবা alternative encoding ব্যবহার করা হয়

Summary
One Hot Encoding হলো Nominal data encode করার সবচেয়ে safe এবং widely used technique। এটি প্রতিটি category কে আলাদা binary column এ রূপান্তর করে, যার ফলে model কোনো ভুল ordering ধরে নেয় না এবং data cleanভাবে represent হয়।

Encoding the Cabin Features

Cabin feature সাধারণত dataset এর একটি খুব গুরুত্বপূর্ণ কিন্তু noisy feature। বিশেষ করে Titanic dataset এর মতো বাস্তব ডেটায় Cabin এর মান অনেক সময় full structure আকারে থাকে, যেমন deck এবং room number একসাথে থাকে।
এই কারণে সরাসরি Cabin কে encoding করা যায় না। আগে এটাকে preprocess করে meaningful feature তৈরি করতে হয়।
Cabin Feature এর সমস্যা বোঝা
প্রথমে Cabin feature এর মান দেখি:

```
1X_train['Cabin'].value_counts()
```

এখানে দেখা যায়:

অনেক missing value থাকতে পারে
অনেক unique cabin value থাকতে পারে
format আলাদা হতে পারে (যেমন C85, E46, B57 ইত্যাদি)

Data type দেখলে:

```
1X_train['Cabin'].dtype
```

সাধারণত এটি object type হয়।
Cabin থেকে গুরুত্বপূর্ণ তথ্য বের করা
Cabin feature সাধারণত এমন format এ থাকে:

C85
E46
B57
D20

এখানে প্রথম letter সাধারণত deck represent করে।
তাই আমরা শুধু প্রথম character বের করব:

```
1X_train['Cabin_Deck'] = X_train['Cabin'].astype(str).str[0]
```

এখানে:

astype(str) → সব value কে string এ convert করে
str[0] → প্রথম character বের করে (deck information)

এখন dataset এ নতুন feature তৈরি হবে:

Cabin_Deck

Cabin Deck Analysis
নতুন feature এর distribution দেখি:

```
1X_train['Cabin_Deck'].value_counts()
```

এখানে আমরা দেখতে পাবো:

A, B, C, D, E, F, G এর মতো deck values
কিছু missing বা unknown category থাকতে পারে

Test Data তে একই **processing**
Training এবং test data একইভাবে process করা খুব গুরুত্বপূর্ণ।

```
1X_test['Cabin_Deck'] = X_test['Cabin'].astype(str).str[0]
```

Cabin Deck Encoding (One Hot Encoding ব্যবহার করে**)**
Cabin_Deck একটি nominal categorical feature, তাই এখানে One Hot Encoding ব্যবহার করা হবে।
প্রথমে encoder তৈরি করি:

```
1from sklearn.preprocessing import OneHotEncoder
2

3deck_ohe = OneHotEncoder(sparse_output=False,
drop='first').set_output(transform='pandas')
```

এখানে:

sparse_output=False → full dataframe output দেয়
drop='first' → dummy variable trap avoid করার জন্য প্রথম category drop করে

Encoder fit করা

```
deck_ohe.fit(X_train[['Cabin_Deck']])
```

এখানে training data থেকে unique deck categories শেখা হয়।

Training Data Transform করা

```
encoded_df = deck_ohe.transform(X_train[['Cabin_Deck']])
```

```
X_train = pd.concat([X_train, encoded_df], axis=1)
```

এখানে:

- Cabin_Deck কে multiple binary columns এ convert করা হয়
- তারপর মূল dataset এর সাথে merge করা হয়

Test Data Transform করা

```
encoded_df = deck_ohe.transform(X_test[['Cabin_Deck']])
```

```
X_test = pd.concat([X_test, encoded_df], axis=1)
```

গুরুত্বপূর্ণ বিষয়:

- fit শুধুমাত্র training data তে হবে
- transform train এবং test দুই জায়গায় ব্যবহার হবে

Unnecessary Column Drop করা

Encoding শেষ হলে original Cabin_Deck column আর দরকার হয় না।

```
X_train.drop('Cabin_Deck', axis=1, inplace=True)
```

```
X_test.drop('Cabin_Deck', axis=1, inplace=True)
```

Final Dataset Structure

Encoding শেষে dataset এ Cabin feature এখন এইভাবে represent হয়:

- Cabin_Deck_A
- Cabin_Deck_B
- Cabin_Deck_C
- Cabin_Deck_D
- Cabin_Deck_E
- Cabin_Deck_F
- Cabin_Deck_G

প্রতিটি column এ value হবে:

- 1 → যদি passenger সেই deck এ থাকে
- 0 → যদি না থাকে

Cabin Feature Encoding এর গুরুত্ব

Cabin directly use করা যায় না কারণ এটি noisy এবং complex
Feature engineering করে meaningful part (deck) বের করতে হয়
তারপর encoding করতে হয়
এটি model performance improve করে

Summary
Cabin feature encoding একটি real-world feature engineering example। এখানে raw text data কে প্রথমে meaningful part (deck) এ convert করা হয়, তারপর One Hot Encoding ব্যবহার করে machine learning model এর জন্য suitable numerical format তৈরি করা হয়।

Label Encoding

Label Encoding হলো এমন একটি encoding technique যেখানে categorical values কে numerical labels এ রূপান্তর করা হয়, সাধারণত 0 থেকে শুরু করে ক্রমানুসারে integer আকারে।
এই technique সাধারণত target variable (y) encoding করার জন্য বেশি ব্যবহার করা হয়, তবে কিছু ক্ষেত্রে feature encoding এর জন্যও ব্যবহার করা যেতে পারে।

Label Encoding কীভাবে কাজ করে
ধরা যাক আমাদের target variable হলো:

Class
Yes
No
Yes
No

Label Encoding করার পর এটি এমন হবে:

Class
1
0
1
0

এখানে প্রতিটি category কে একটি unique integer দেওয়া হয়েছে।

Label Encoding কেন ব্যবহার করা হয়

Machine Learning model সাধারণত numerical target চায়
Binary classification problem এ খুব সহজ এবং effective
Memory efficient (কম column তৈরি হয় না)
Simple encoding process

Label Encoding এর সীমাবদ্ধতা

Label Encoding feature হিসেবে ব্যবহার করলে কিছু সমস্যা হতে পারে:

Model ভুলভাবে order ধরে নিতে পারে
Nominal feature এ inappropriate relationship তৈরি হতে পারে

তাই সাধারণত:

Target variable (y) এর জন্য best
Feature encoding এর জন্য সাবধানে ব্যবহার করতে হয়

Scikit-Learn দিয়ে **Label Encoding**

প্রথমে প্রয়োজনীয় library import করি:

```
1from sklearn.preprocessing import LabelEncoder
```

LabelEncoder object তৈরি করি:

```
1le = LabelEncoder()
```

Fit করা

```
1le.fit(y_train)
```

এখানে training target variable থেকে unique classes শেখা হয়।

Transform করা

```
1y_train_array = le.transform(y_train)
```

```
2y_test_array = le.transform(y_test)
```

এখানে:

y_train এবং y_test কে numerical format এ convert করা হচ্ছে
একই mapping ব্যবহার করা হচ্ছে যাতে consistency বজায় থাকে

Array থেকে **Series** এ রূপান্তর

LabelEncoder output সাধারণত numpy array দেয়। তাই আবার pandas Series এ convert করা হয়:

```
1y_train = pd.Series(y_train_array, index=y_train.index,
name=y_train.name)
```

```
2y_test = pd.Series(y_test_array, index=y_test.index, name=y_test.name)
```

এখানে:

original index preserve করা হয়েছে
original column name রাখা হয়েছে
data structure আবার pandas compatible করা হয়েছে

Final Output Example

ধরা যাক:

Original y
Yes
No
Yes

Encoding এর পর:

Encoded y
1
0
1

Label Encoding এর সুবিধা

- খুব সহজ এবং fast
- Binary classification এ খুব effective
- কম memory ব্যবহার করে
- Target variable encoding এর জন্য standard approach

Label Encoding এর অসুবিধা

- Feature হিসেবে ব্যবহার করলে misleading order তৈরি হতে পারে
- Multi-class nominal feature এ ভুল interpretation হতে পারে

Summary

Label Encoding হলো একটি simple কিন্তু powerful encoding technique, যা মূলত target variable কে numerical format এ রূপান্তর করার জন্য ব্যবহৃত হয়। এটি machine learning pipeline এ খুব গুরুত্বপূর্ণ ভূমিকা পালন করে, বিশেষ করে classification problems এ।